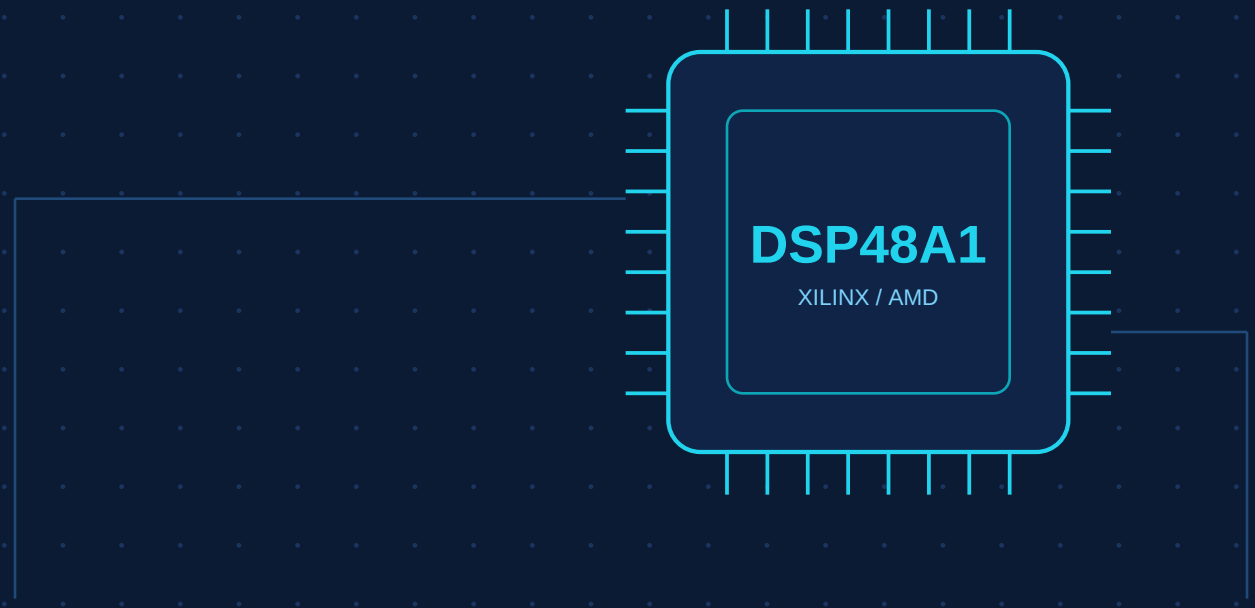


DIGITAL DESIGN PROJECT

DSP48A1 Slice

Verilog Design & Implementation

RTL, testbench, simulation, elaboration, synthesis, implementation and linting results.

TARGET	LANGUAGE	TOOLS	DEVICE
Xilinx / AMD DSP48A1	Verilog HDL	QuestaSim · Vivado	xc7a200tffg1156-3

Contents

01 The DSP48A1 Slice . . . 3
- Architecture and data flow . . . 4
- Parameters (attributes) . . . 4
- OPMODE pin descriptions . . . 5
- Data and cascade ports . . . 5
- Registers, clock enables and resets . . . 6

02 Design Overview . . . 7

03 Simulation Waveforms . . . 8

04 RTL Code . . . 10
- register . . . 10
- add_sub and multiplier . . . 11
- pipline_mux . . . 12
- input_data . . . 12
- output_data . . . 13
- pro_1 (top module) . . . 14

05 Testbench . . . 15
- Test plan . . . 15
- Test 2.1: Verify reset operation . . . 16
- Test 2.2: DSP path 1 (pre-subtracter, multiplier, post-subtracter) . . . 16
- Test 2.3: DSP path 2 (pre-adder, zeros into the post-adder) . . . 17
- Test 2.4: DSP path 3 (P feedback) . . . 18
- Test 2.5: DSP path 4 (D:A:B concatenation and PCIN) . . . 19
- Testbench code, part 1: setup and reset test (2.1) . . . 20
- Testbench code, part 2: directed tests 2.2 – 2.4 . . . 21
- Testbench code, part 3: test 2.5 and end of testbench . . . 22

06 Elaboration . . . 23

07 Synthesis . . . 24

08 Implementation . . . 26

09 Linting . . . 27

01 The DSP48A1 Slice

What the block is and how it works

The Spartan-6 family offers a high ratio of DSP48A1 slices to logic, which makes it well suited to math-intensive applications. Each slice combines an 18-bit **pre-adder/subtracter**, an 18 × 18 **multiplier** and a 48-bit **post-adder/subtracter**, with an optional pipeline register after almost every stage. Its behaviour is set in two ways: by **parameters** fixed at configuration time, and by the 8-bit **OPMODE** input, which can change while the slice is running.

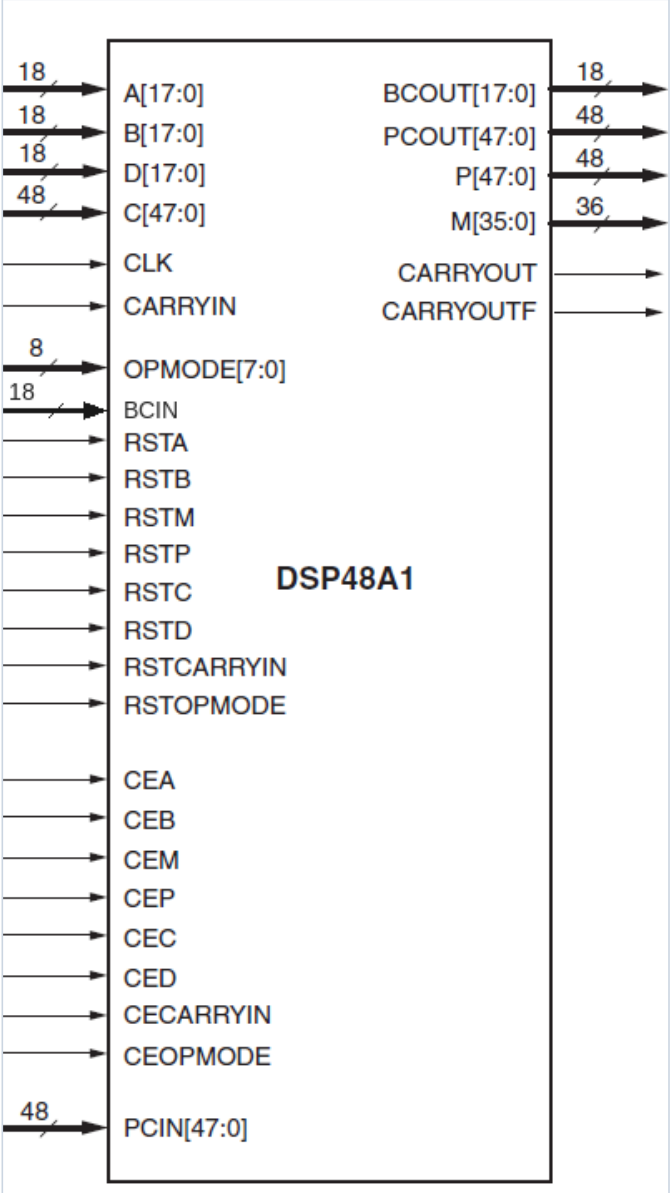


Figure 1. DSP48A1 port diagram.

Ports at a glance

Group	Signals
Data in	A, B, D (18 bits), C (48 bits)
Cascade in	BCIN (18), PCIN (48)
Carry in	CARRYIN
Control	CLK, OPMODE[7:0]
Clock enables	CEA, CEB, CEC, CED, CEM, CEP, CECARRYIN, CEOPMODE
Resets	RSTA, RSTB, RSTC, RSTD, RSTM, RSTP, RSTCARRYIN, RSTOPMODE
Data out	P (48), M (36)
Cascade out	BCOUT (18), PCOUT (48)
Carry out	CARRYOUT, CARRYOUTF

Architecture and data flow

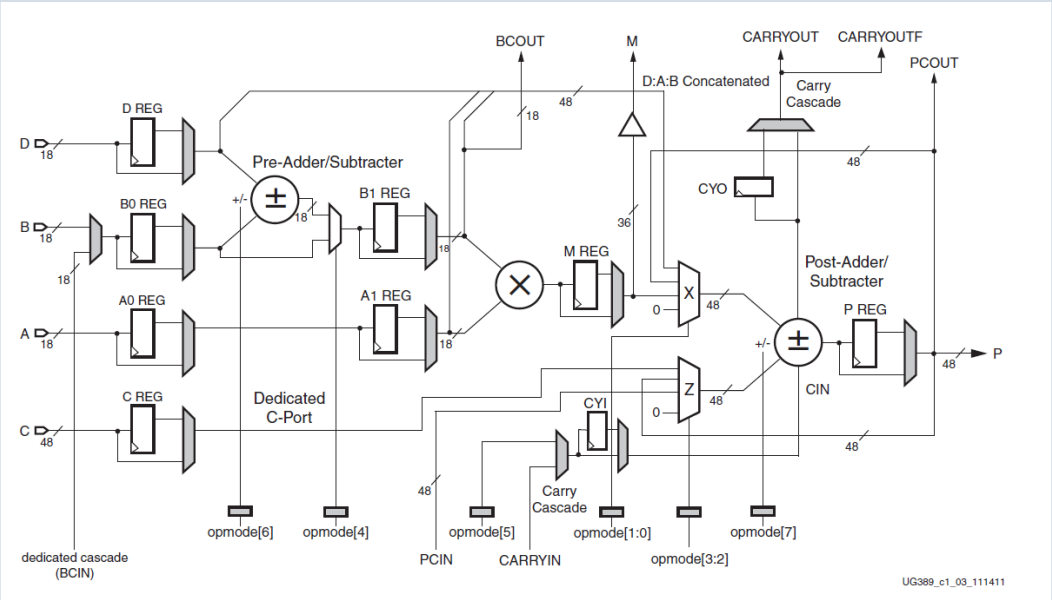


Figure 2. DSP48A1 architecture (Xilinx UG389).

- D and B enter the **pre-adder/subtracter**: D − B when OPMODE[6] = 1, otherwise D + B. OPMODE[4] decides whether its result or B itself feeds the multiplier.
- The **multiplier** produces the 36-bit product, available on M (registered when MREG = 1).
- The **X multiplexer** (OPMODE[1:0]) selects 0, the product, P (accumulate) or the concatenation D:A:B. The **Z multiplexer** (OPMODE[3:2]) selects 0, PCIN, P or C.
- The **post-adder/subtracter** computes Z + X + CIN, or Z − (X + CIN) when OPMODE[7] = 1, and drives P, CARRYOUT and CARRYOUTF.
- D:A:B is concatenated as D[11:0], A[17:0], B[17:0]. X and Z are 48 bits wide, so narrower inputs are zero-extended.
- Grey multiplexers are fixed by parameters at configuration time. Clear multiplexers are driven by OPMODE and can change dynamically.

Parameters (attributes)

Parameter	Values (default)	Function
A0REG, A1REG, B0REG, B1REG	0 or 1 (A0REG = 0, A1REG = 1, B0REG = 0, B1REG = 1)	Number of pipeline registers on the A and B paths. A0 and B0 are the first stage, A1 and B1 the second.
CREG, DREG, MREG, PREG, CARRYINREG, CARRYOUTREG, OPMODEREG	0 or 1 (default 1)	Number of pipeline stages: 1 = registered, 0 = bypassed.
CARRYINSEL	"CARRYIN" or "OPMODE5" (default "OPMODE5")	Carry-cascade source: the CARRYIN pin or OPMODE[5]. Any other string ties the mux output to 0.
B_INPUT	"DIRECT" or "CASCADE" (default "DIRECT")	B comes from the B port (DIRECT) or from BCIN of the previous slice (CASCADE). Any other string ties it to 0.
RSTTYPE	"SYNC" or "ASYNC" (default "SYNC")	Synchronous or asynchronous reset for all registers in the slice.

OPMODE pin descriptions

Bits	Selects	Value: meaning
`OPMODE[1:0]`	Source of X	0: zeros (disables the post-adder, Z passes to P) 1: multiplier product 2: P output (accumulator) 3: concatenated D:A:B
`OPMODE[3:2]`	Source of Z	0: zeros (the product or other X result passes to P) 1: PCIN 2: P output (accumulator) 3: C port
`OPMODE[4]`	Pre-adder use	0: bypass, B goes straight to the multiplier 1: use the pre-adder/subtracter on B and D before the multiplier
`OPMODE[5]`	Carry input	Forces the value on the carry input of CYI, or direct to CIN. Only applies when CARRYINSEL = "OPMODE5".
`OPMODE[6]`	Pre-adder operation	0: addition 1: subtraction (D − B)
`OPMODE[7]`	Post-adder operation	0: addition 1: subtraction (Z − (X + CIN))

Data and cascade ports

Signal	Bits	Function
`A`	18	Multiplier input, and optionally a post-adder input through OPMODE[1:0].
`B`	18	Pre-adder input; reaches the multiplier through OPMODE[4] or the post-adder through OPMODE[1:0].
`C`	48	Post-adder/subtracter input.
`D`	18	Pre-adder input. D[11:0] is concatenated with A and B for the post-adder.
`CARRYIN`	1	Carry input to the post-adder/subtracter.
`M`	36	Multiplier output: the M register when MREG = 1, otherwise the direct product.
`P`	48	Post-adder output: the P register when PREG = 1, otherwise the direct result.
`CARRYOUT`	1	Cascade carry out (registered when CARRYOUTREG = 1). Connects only to CARRYIN of an adjacent slice.
`CARRYOUTF`	1	Copy of CARRYOUT for use in the FPGA logic.
`BCIN / BCOUT`	18	Cascade input from the adjacent slice’s BCOUT / cascade output for port B.
`PCIN / PCOUT`	48	Cascade input / output for port P.

Registers, clock enables and resets

All resets are active high and are synchronous or asynchronous depending on RSTTYPE. CLK is the DSP clock and OPMODE selects the arithmetic operation.

Register	Clock enable	Reset
A registers (A0REG, A1REG)	CEA	RSTA
B registers (B0REG, B1REG)	CEB	RSTB
C register (CREG)	CEC	RSTC
D register (DREG)	CED	RSTD
Multiplier register (MREG)	CEM	RSTM
P output register (PREG)	CEP	RSTP
Carry-in and carry-out registers (CYI, CYO)	CECARRYIN	RSTCARRYIN
OPMODE register (OPMODEREG)	CEOPMODE	RSTOPMODE

02 Design Overview

A parameterised Verilog model of the DSP48A1 slice

This report documents a parameterised Verilog implementation of the Xilinx (AMD) **DSP48A1** slice: an input stage with optional pipeline registers and a pre-adder, an 18 × 18 multiplier, X/Z operand multiplexers and a 48-bit post-adder. It collects the RTL, the testbench, the QuestaSim waveforms and the Vivado elaboration, synthesis, implementation and linting results, in that order.

Module hierarchy

Module	Role
`pro_l`	Top level. Multiplier, M and CARRYIN registers, X/Z multiplexers, carry-in select.
`input_data`	Input stage: D, B0, C, A0 and OPMODE registers, pre-adder/subtractor, B1 and A1 registers.
`output_data`	Output stage: post-adder/subtractor, P and CARRYOUT registers.
`multiplier`	Combinational WIDTH × WIDTH multiplier with a 2·WIDTH-bit product.
`add_sub`	Adder/subtractor with carry in and carry/borrow out.
`pipline_mux`	Selects the registered or the bypassed (unregistered) path via the PIPLINE parameter.
`register`	Register with enable and synchronous or asynchronous reset.

Key parameters (top level)

Parameter	Default	Parameter	Default
WIDTH_ABD	18	WIDTH_P	48
WIDTH_C	48	WIDTH_M	36
WIDTH_OP	8	B_INPUT	"DIRECT" (or "CASCADE")
RSTTYPE	"SNYC" (or "ASYNC")	CARRYINSEL	"OPMODE5" (or "CARRYIN")

Results at a glance

Metric	Post-synthesis	Post-implementation
Slice LUTs	273	273
Slice registers	160	179
DSP blocks	1	1
Worst negative slack (WNS)	5.216 ns	3.716 ns
Worst hold slack (WHS)	0.182 ns	0.262 ns
Worst pulse width slack (WPWS)	4.500 ns	4.500 ns
Failing endpoints	0	0

Clock constraint: 10.00 ns period on port `clk`. All user-specified timing constraints are met in both stages.

03 Simulation Waveforms

QuestaSim wave window, 0 to 198 ns

Waveform, 0 – 99 ns

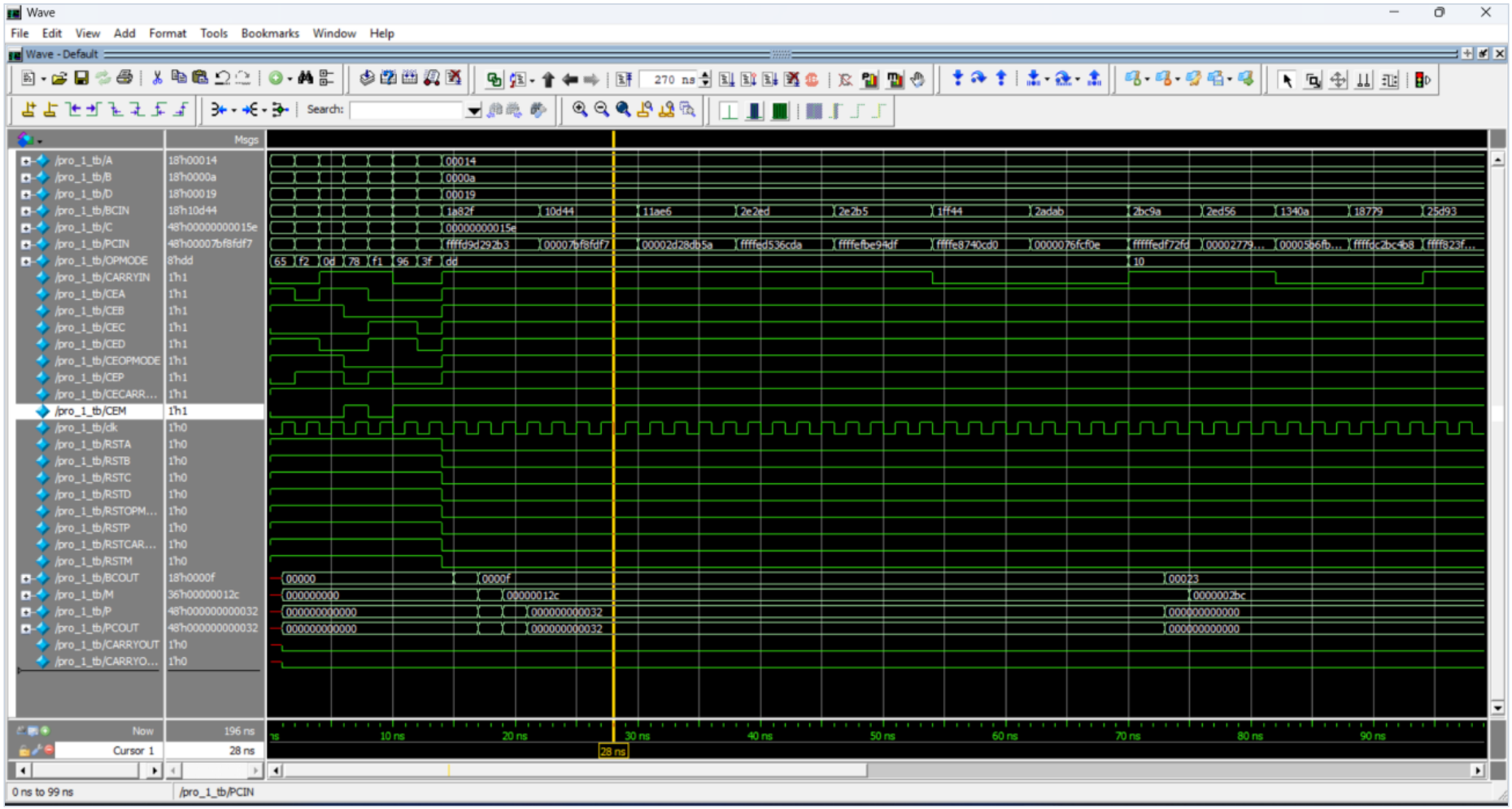


Figure 3. QuestaSim waveform, 0 – 99 ns.

Waveform, 99 – 198 ns

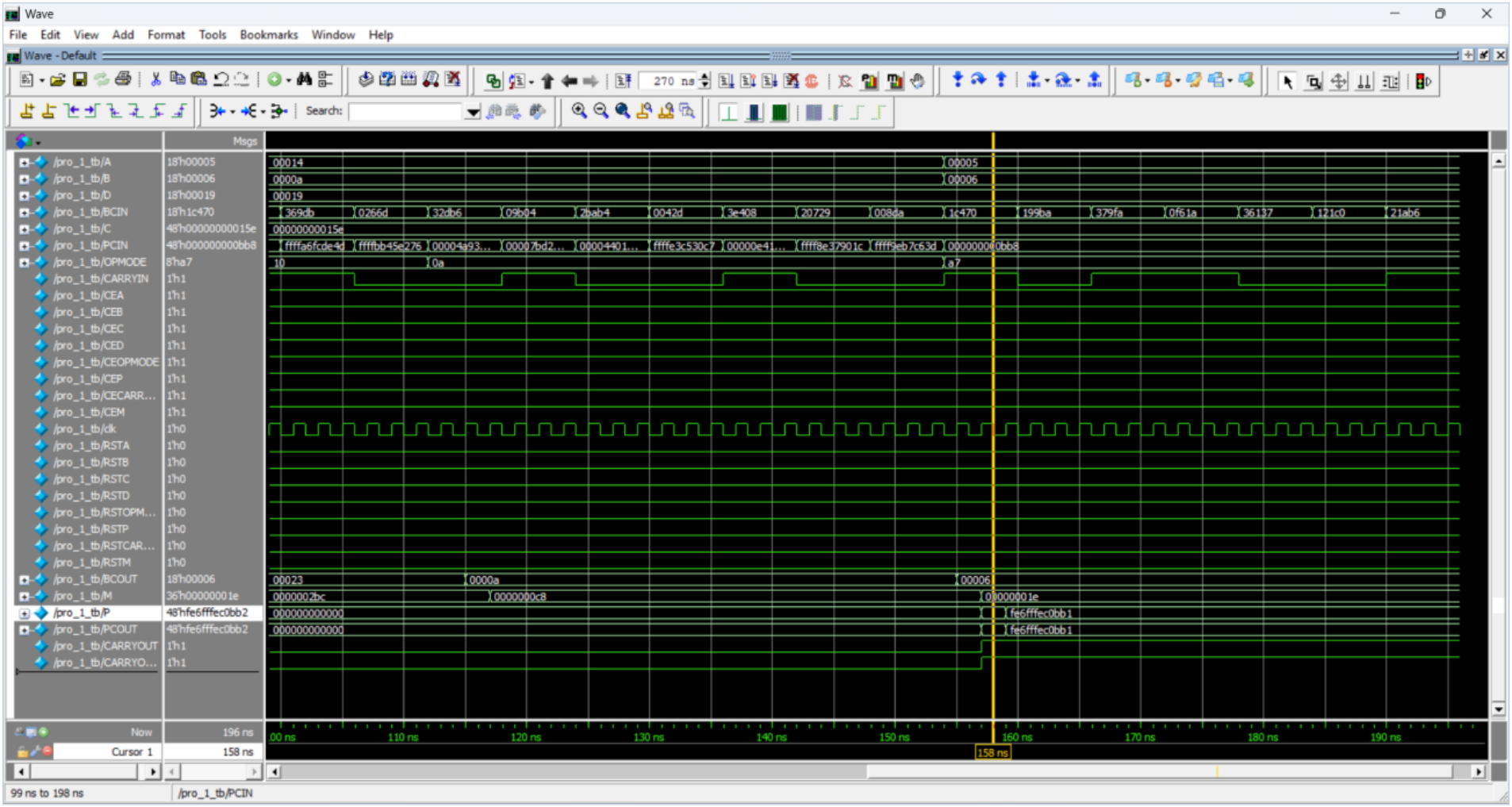


Figure 4. *QuestaSim waveform, 99 – 198 ns.*

04 RTL Code

Verilog source for every module, bottom-up

register

Register with clock enable. RSTTYPE selects a synchronous or an asynchronous reset through a generate-if.

```verilog
module register (in,out,e,clk,rst );
    parameter WIDTH =1;
    parameter RSTTYPE = "SNYC"; //OR "ASYNC"
    input clk,rst,e;
    input [WIDTH-1:0] in ;
    output reg [WIDTH-1:0] out;
    generate
        if (RSTTYPE == "SNYC") begin
            always @(posedge clk) begin
                if (rst) begin
                    out <= 'b0;
                end else if(e) begin
                    out <= in;
                end
            end
        end else begin
            always @(posedge clk or posedge rst) begin
                if (rst) begin
                    out <= 'b0;
                end else if(e) begin
                    out <= in;
                end
            end
        end
    endgenerate
endmodule //register
```

Figure 5. `register` *module.*

add_sub and multiplier

Adder/subtractor (sel = 0: in1 + in2 + cin, sel = 1: in1 − (in2 + cin)) and a WIDTH × WIDTH multiplier.

```
1  module add_sub (in1,in2,cin,sel,out,cout);
2      parameter WIDTH =1;
3      input [WIDTH-1:0] in1,in2;
4      input cin,sel;
5      output reg [WIDTH-1:0] out;
6      output reg cout;
7      always @(*) begin
8          if (sel) begin
9              {cout,out}=in1-(in2+cin);
10         end else begin
11             {cout,out}=in1+in2+cin;
12         end
13     end
14 endmodule //add_sub
15 module multiplier (in1,in2,out);
16     parameter WIDTH =1;
17     localparam OUT_W = WIDTH * 2 ;
18     input [WIDTH-1:0] in1,in2;
19     output [OUT_W-1 : 0] out;
20     assign out = in1 * in2 ;
21 endmodule //multiplier
```

Figure 6. `add_sub` *and* `multiplier` *modules.*

pipline_mux

Register plus bypass multiplexer: PIPLINE = 1 selects the registered value, otherwise the input passes straight through.

```
module pipline_mux (in1,out,e,clk,rst );
    parameter WIDTH =1;
    parameter RSTTYPE = "SNYC"; //OR "ASYNC"
    parameter PIPLINE = 1;
    input clk,rst,e;
    input [WIDTH-1:0] in1;
    output [WIDTH-1:0] out;
    wire [WIDTH-1:0] in1_pip_mux;
    register #(.WIDTH(WIDTH),.RSTTYPE(RSTTYPE)) reg_pip (in1,in1_pip_mux,e,clk,rst);
    assign out = (PIPLINE==1) ? in1_pip_mux : in1;
endmodule //pipline_mux
```

Figure 7. `pipline_mux` *module.*

input_data

Input stage: registers D, B0, C, A0, OPMODE, B1 and A1, and the pre-adder/subtractor (OPMODE[6] selects the operation, OPMODE[4] selects the B1 source).

```
module input_data #(
    parameter WIDTH_ABD = 18,
    parameter WIDTH_C = 48,
    parameter WIDTH_OP = 8,
    parameter RSTTYPE = "SNYC", //OR "ASYNC"
    parameter A0_REG = 0,
    parameter B0_REG = 0,
    parameter C_REG = 1,
    parameter D_REG = 1,
    parameter A1_REG = 1,
    parameter B1_REG = 1,
    parameter OPMODE_REG = 1,
    parameter B_INPUT = "DIRECT" //OR "CASCADE"
    )(
    input [WIDTH_ABD-1:0] A,B,D,BCIN,
    input [WIDTH_C-1:0] C,
    input [WIDTH_OP-1:0] OPMODE,
    input CEA,CEB,CEC,CED,CEOPMODE,clk,RSTA,RSTB,RSTC,RSTD,RSTOPMODE,
    output [WIDTH_ABD-1:0] out_d,out_b1,out_a1,
    output [WIDTH_C-1:0] out_c,
    output [WIDTH_OP -1 :0]  out_op
    );
    wire [WIDTH_ABD-1:0] in_b0,out_b0,out_a0,out_as,in_b1;
    assign in_b0 = (B_INPUT =="DIRECT") ? B : (B_INPUT =="CASCADE") ? BCIN : 'b0;
    pipline_mux #(.WIDTH(WIDTH_ABD),.RSTTYPE(RSTTYPE),.PIPLINE(D_REG)) d_reg_mux (D,out_d,CED,clk,RSTD);
    pipline_mux #(.WIDTH(WIDTH_ABD),.RSTTYPE(RSTTYPE),.PIPLINE(B0_REG)) b0_reg_mux (in_b0,out_b0,CEB,clk,RSTB);
    pipline_mux #(.WIDTH(WIDTH_ABD),.RSTTYPE(RSTTYPE),.PIPLINE(A0_REG)) a0_reg_mux (A,out_a0,CEA,clk,RSTA);
    pipline_mux #(.WIDTH(WIDTH_C),.RSTTYPE(RSTTYPE),.PIPLINE(C_REG)) c_reg_mux (C,out_c,CEC,clk,RSTC);
    pipline_mux #(.WIDTH(WIDTH_OP),.RSTTYPE(RSTTYPE),.PIPLINE(OPMODE_REG)) op_reg_mux (OPMODE,out_op,CEOPMODE,clk,RSTOPMODE);
    add_sub #(.WIDTH(WIDTH_ABD)) add_sub_1(.in1(out_d),.in2(out_b0),.cin(1'b0),.sel(out_op[6]),.out(out_as),.cout());
    assign  in_b1 = (out_op[4])?out_as:out_b0;
    pipline_mux #(.WIDTH(WIDTH_ABD),.RSTTYPE(RSTTYPE),.PIPLINE(B1_REG)) b1_reg_mux (in_b1,out_b1,CEB,clk,RSTB);
    pipline_mux #(.WIDTH(WIDTH_ABD),.RSTTYPE(RSTTYPE),.PIPLINE(A1_REG)) a1_reg_mux (out_a0,out_a1,CEA,clk,RSTA);
endmodule //input_data
```

Figure 8. `input_data` *module.*

output_data

Output stage: post-adder/subtractor on the X and Z operands (OPMODE[7] selects the operation), then the P and CARRYOUT registers. CARRYOUTF mirrors CARRYOUT.

```verilog
module output_data #(
    parameter WIDTH_P = 48,
    parameter RSTTYPE = "SNYC", //OR "ASYNC"
    parameter CARRYOUT_REG = 1,
    parameter P_REG = 1
    )(
    input [WIDTH_P-1:0] out_z,out_x,
    input sel,cin,CEP,CECARRYIN,clk,RSTP,RSTCARRYIN,
    output [WIDTH_P-1:0] P,
    output CARRYOUT,CARRYOUTF
    );
    wire [WIDTH_P-1:0] in_p;
    wire in_cyo;
    assign CARRYOUTF = CARRYOUT;
    add_sub #(.WIDTH(WIDTH_P)) add_sub_2(.in1(out_z),.in2(out_x),.cin(cin),.sel(sel),.out(in_p),.cout(in_cyo));
    pipline_mux #(.WIDTH(WIDTH_P),.RSTTYPE(RSTTYPE),.PIPLINE(P_REG)) p_reg_mux (in_p,P,CEP,clk,RSTP);
    pipline_mux #(.RSTTYPE(RSTTYPE),.PIPLINE(CARRYOUT_REG)) cyo_reg_mux (in_cyo,CARRYOUT,CECARRYIN,clk,RSTCARRYIN);
endmodule //output_data
```

Figure 9. `output_data` *module.*

pro_1 (top module)

Wires everything together. X mux: OPMODE[1:0]; Z mux: OPMODE[3:2]; carry-in source: CARRYINSEL (OPMODE[5] or the CARRYIN pin).

```
module pro_1 #(
    parameter WIDTH_ABD = 18,
    parameter WIDTH_C = 48,
    parameter WIDTH_OP = 8,
    parameter WIDTH_P = 48,
    parameter WIDTH_M = 36,
    parameter RSTTYPE = "SNYC", //OR "ASYNC"
    parameter A0_REG = 0,
    parameter B0_REG = 0,
    parameter C_REG = 1,
    parameter D_REG = 1,
    parameter A1_REG = 1,
    parameter B1_REG = 1,
    parameter OPMODE_REG = 1,
    parameter M_REG = 1,
    parameter CARRYIN_REG = 1,
    parameter CARRYOUT_REG = 1,
    parameter P_REG = 1 ,
    parameter B_INPUT = "DIRECT" ,//OR "CASCADE"
    parameter CARRYINSEL= "OPMODE5" //OR "CARRYIN"
    )(
    input [WIDTH_ABD-1:0] A,B,D,BCIN,
    input [WIDTH_C-1:0] C,PCIN,
    input [WIDTH_OP-1:0] OPMODE,
    input CARRYIN,
    input CEA,CEB,CEC,CED,CEOPMODE,CEP,CECARRYIN,CEM,clk,
    RSTA,RSTB,RSTC,RSTD,RSTOPMODE,RSTP,RSTCARRYIN,RSTM,
    output [WIDTH_ABD-1:0] BCOUT,
    output [WIDTH_M-1:0] M,
    output [WIDTH_P-1:0] P,PCOUT,
    output CARRYOUT,CARRYOUTF
    );
    wire [WIDTH_ABD-1:0]out_d,out_a1;
    wire [WIDTH_C-1:0] out_c;
    wire [WIDTH_M-1:0] in_m,out_m;
    wire [WIDTH_OP-1:0] out_op;
    reg [WIDTH_P-1:0] out_z,out_x;
    wire in_cyi , out_cyi;
    input_data #(
        .WIDTH_ABD(WIDTH_ABD),
        .WIDTH_C(WIDTH_C),
        .WIDTH_OP(WIDTH_OP),
        .RSTTYPE(RSTTYPE),
        .A0_REG(A0_REG),
        .B0_REG(B0_REG),
        .C_REG(C_REG),
        .D_REG(D_REG),
        .A1_REG(A1_REG),
        .B1_REG(B1_REG),
        .OPMODE_REG(OPMODE_REG),
        .B_INPUT(B_INPUT)
        ) p_1 (
        A,B,D,BCIN,C,OPMODE,
        CEA,CEB,CEC,CED,CEOPMODE,clk,RSTA,RSTB,RSTC,RSTD,RSTOPMODE,
        out_d,BCOUT,out_a1,out_c,out_op
        );
    multiplier #(.WIDTH(WIDTH_ABD)) multi1 (BCOUT,out_a1,in_m);
    assign in_cyi = (CARRYINSEL == "OPMODE5")? out_op[5]:(CARRYINSEL == "CARRYIN")? CARRYIN :'b0;
    pipline_mux #(.RSTTYPE(RSTTYPE),.PIPLINE(CARRYIN_REG)) cyi_reg_mux (in_cyi,out_cyi,CECARRYIN,clk,RSTCARRYIN);
    pipline_mux #(.WIDTH(WIDTH_M),.RSTTYPE(RSTTYPE),.PIPLINE(M_REG)) m_reg_mux (in_m,out_m,CEM,clk,RSTM);
    //x
    always @(*) begin
        case (out_op[1:0])
            2'b00: out_x= 'b0;
            2'b01: out_x={12'b0,out_m};
            2'b10: out_x=P;
            2'b11: out_x= {out_d[11:0],out_a1[17:0],BCOUT[17:0]};
        endcase
    end
    //z
    always @( *) begin
        case (out_op[3:2])
            2'b00: out_z= 'b0;
            2'b01: out_z=PCIN;
            2'b10: out_z=P;
            2'b11: out_z=out_c;
        endcase
    end
    output_data #(
    .WIDTH_P (WIDTH_P),
    .RSTTYPE (RSTTYPE),
    .CARRYOUT_REG(CARRYOUT_REG),
    .P_REG(P_REG)
    )p_2(
    out_z,out_x,
    out_op[7],out_cyi,CEP,CECARRYIN,clk,RSTP,RSTCARRYIN,
    P,CARRYOUT,CARRYOUTF
    );
    assign M =out_m;
    assign PCOUT = P;
endmodule //pro_1
```

Figure 10. `pro_1` *top-level module.*

05 Testbench

Test plan (2.1 – 2.5) and testbench code

Test plan

The slice runs with its default parameters throughout. Test 2.1 checks the resets; tests 2.2 to 2.5 each drive a different path through the slice by changing OPMODE and compare every output with its expected value, printing a message on any mismatch.

Test	OPMODE	Inputs	Wait	Expected outputs
2.1 Reset	any	All resets = 1, other inputs random	1 negedge	All outputs = 0
2.2 Path 1	8'b11011101	A = 20, B = 10, C = 350, D = 25	4 negedges	BCOUT = 'hf, M = 'h12c, P = PCOUT = 'h32, CARRYOUT = CARRYOUTF = 0
2.3 Path 2	8'b00010000	A = 20, B = 10, C = 350, D = 25	3 negedges	BCOUT = 'h23, M = 'h2bc, P = PCOUT = 0, CARRYOUT = CARRYOUTF = 0
2.4 Path 3	8'b00001010	A = 20, B = 10, C = 350, D = 25	3 negedges	BCOUT = 'ha, M = 'hc8, P = PCOUT = previous P, CARRYOUT = CARRYOUTF = previous CARRYOUT
2.5 Path 4	8'b10100111	A = 5, B = 6, C = 350, D = 25, PCIN = 3000	3 negedges	BCOUT = 'h6, M = 'h1e, P = PCOUT = 'hfe6fffec0bb1, CARRYOUT = CARRYOUTF = 1

Default configuration

Parameter	Value	Parameter	Value
A0REG	0	CARRYINREG	1
A1REG	1	CARRYOUTREG	1
B0REG	0	OPMODEREG	1
B1REG	1	CARRYINSEL	"OPMODE5"
CREG	1	B_INPUT	"DIRECT"
DREG	1	RSTTYPE	"SYNC"
MREG	1	PREG	1

Test 2.1: Verify reset operation

Assert every active-high reset and drive the remaining inputs with random values. After one negative clock edge all outputs must be zero. The resets are then released and all clock enables asserted so the following tests run on a clean, enabled slice.

Test 2.2: DSP path 1 (pre-subtracter, multiplier, post-subtracter)

Pre-subtracter on, multiplier output routed through mux X and the C port through mux Z, with the post-adder in subtract mode. The data crosses four registers (DREG, B1REG, MREG, PREG) before it reaches P.

OPMODE	8'b11011101
Inputs	A = 20, B = 10, C = 350, D = 25; BCIN, PCIN, CARRYIN random
Wait	4 negative clock edges
Expected	BCOUT = 'hf, M = 'h12c, P = PCOUT = 'h32, CARRYOUT = CARRYOUTF = 0

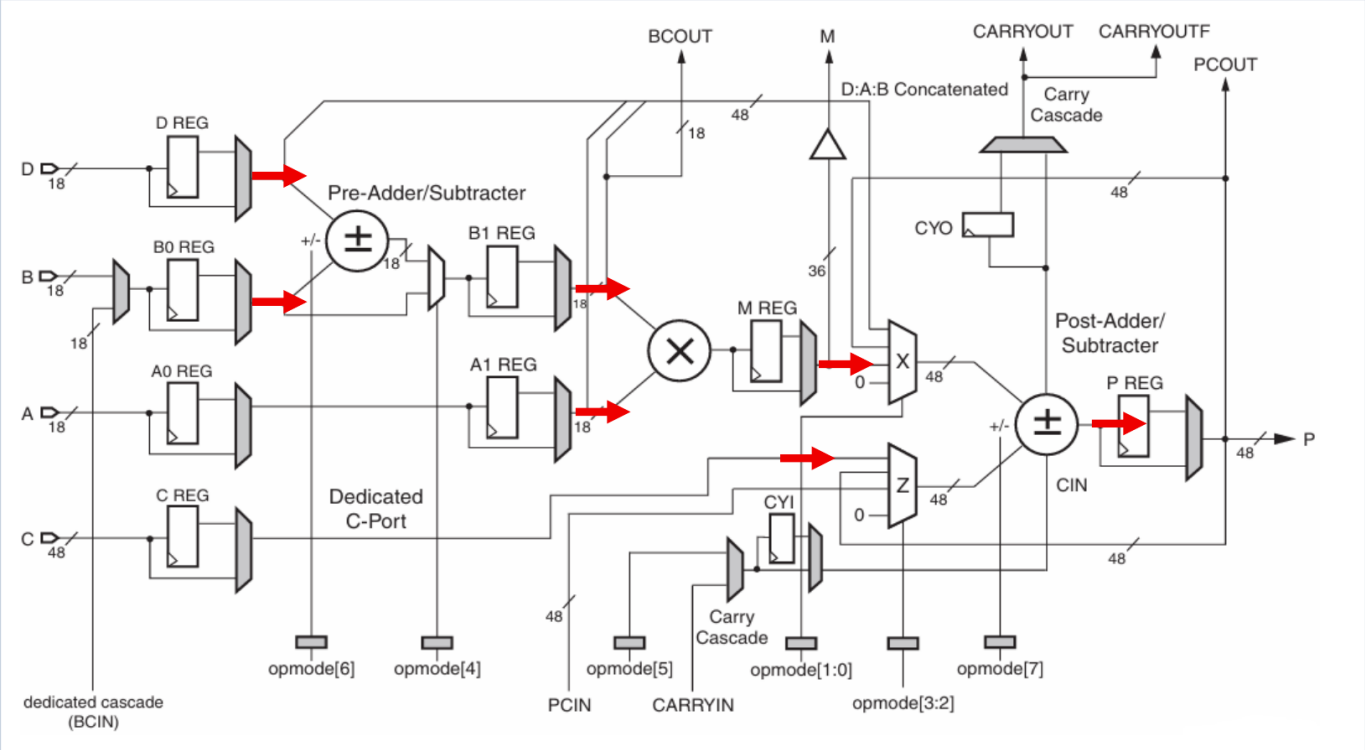


Figure 11. Path 1 data flow (red arrows mark the active path).

Test 2.3: DSP path 2 (pre-adder, zeros into the post-adder)

Pre-adder on and zeros routed through both mux X and mux Z. The longest path crosses three registers (DREG, B1REG, MREG); PREG is loaded in parallel.

OPMODE	8'b00010000
Inputs	A = 20, B = 10, C = 350, D = 25; BCIN, PCIN, CARRYIN random
Wait	3 negative clock edges
Expected	BCOUT = 'h23, M = 'h2bc, P = PCOUT = 0, CARRYOUT = CARRYOUTF = 0

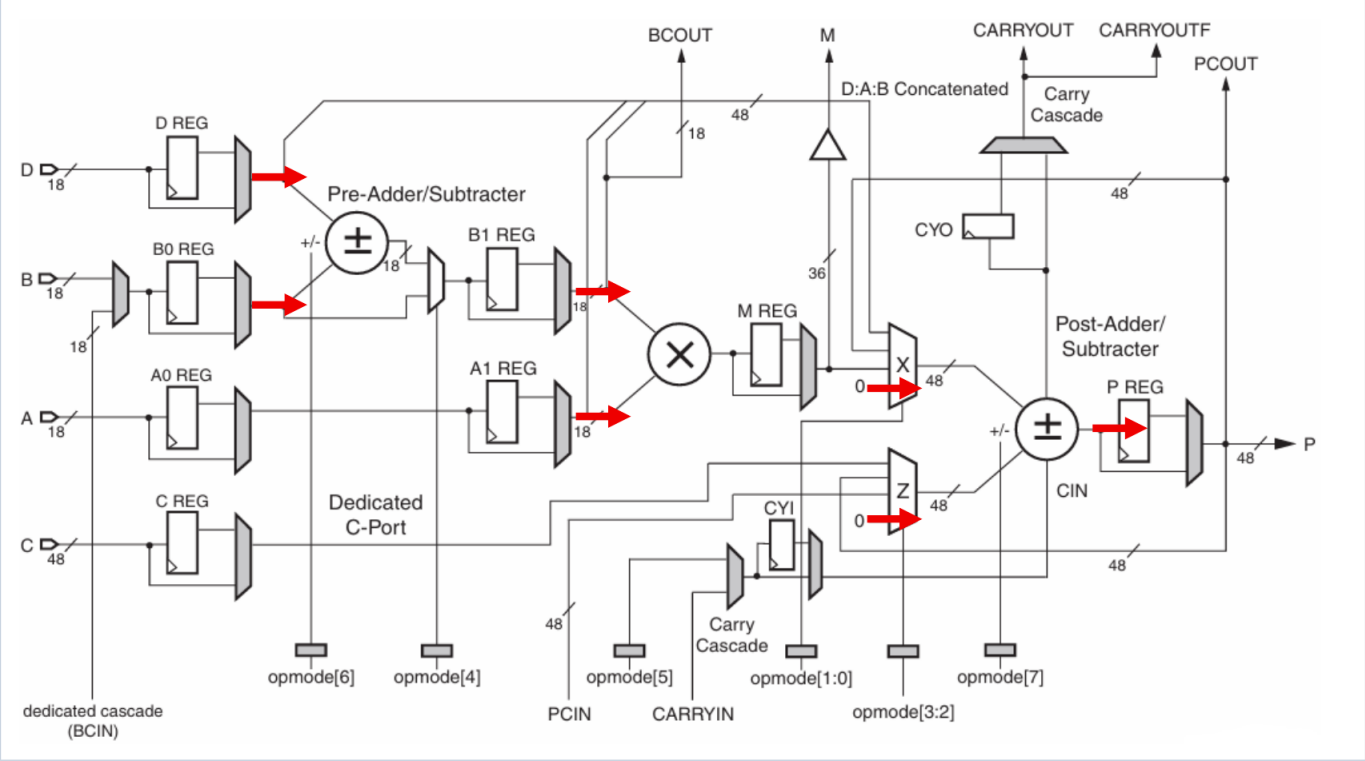


Figure 12. Path 2 data flow.

Test 2.4: DSP path 3 (P feedback)

No pre-adder or pre-subtracter. The P feedback is routed through mux X and mux Z, so the post-adder works on the previous value of P. The longest path crosses three registers (B1REG, MREG, PREG).

OPMODE	8'b00001010
Inputs	A = 20, B = 10, C = 350, D = 25; BCIN, PCIN, CARRYIN random
Wait	3 negative clock edges
Expected	BCOUT = 'ha, M = 'hc8, P = PCOUT = previous P, CARRYOUT = CARRYOUTF = previous CARRYOUT

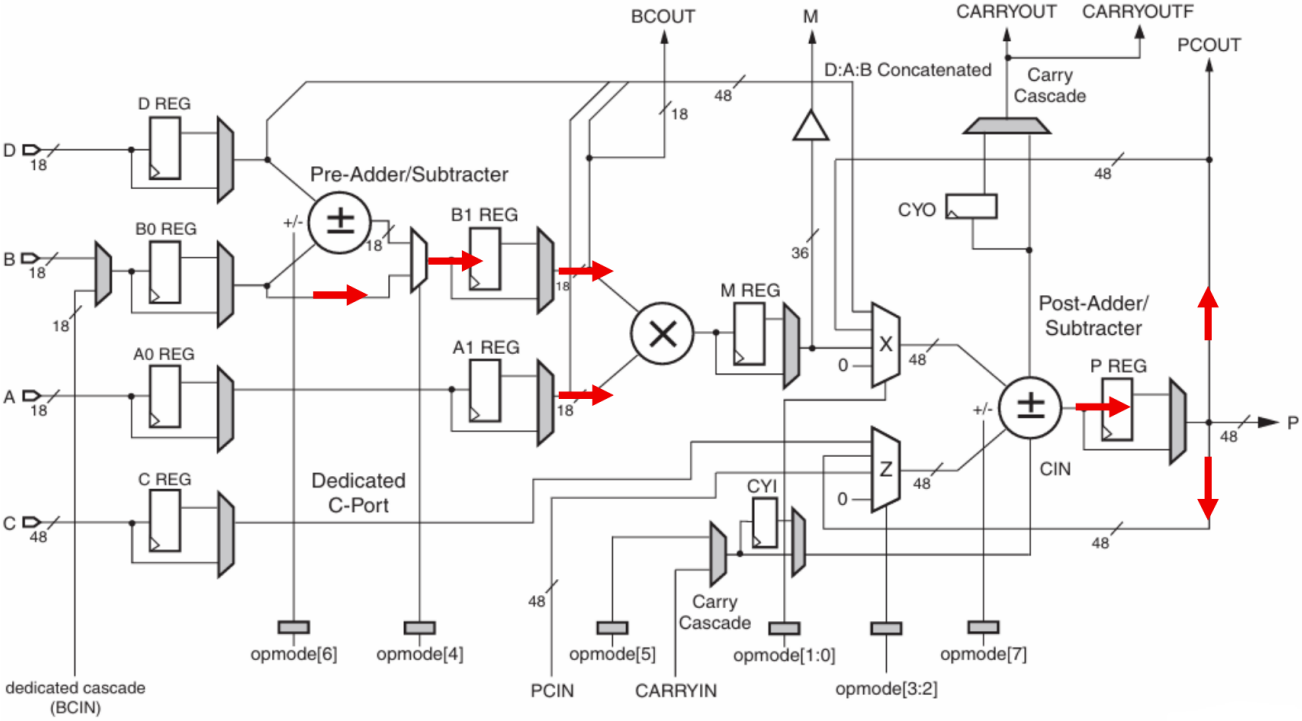


Figure 13. Path 3 data flow.

Test 2.5: DSP path 4 (D:A:B concatenation and PCIN)

No pre-adder or pre-subtracter. The concatenated D:A:B is routed through mux X and PCIN through mux Z, with the post-adder in subtract mode. The longest path crosses three registers (B1REG, MREG, PREG).

OPMODE	8'b10100111
Inputs	A = 5, B = 6, C = 350, D = 25, PCIN = 3000; BCIN, CARRYIN random
Wait	3 negative clock edges
Expected	BCOUT = 'h6, M = 'h1e, P = PCOUT = 'hfe6fffec0bb1, CARRYOUT = CARRYOUTF = 1

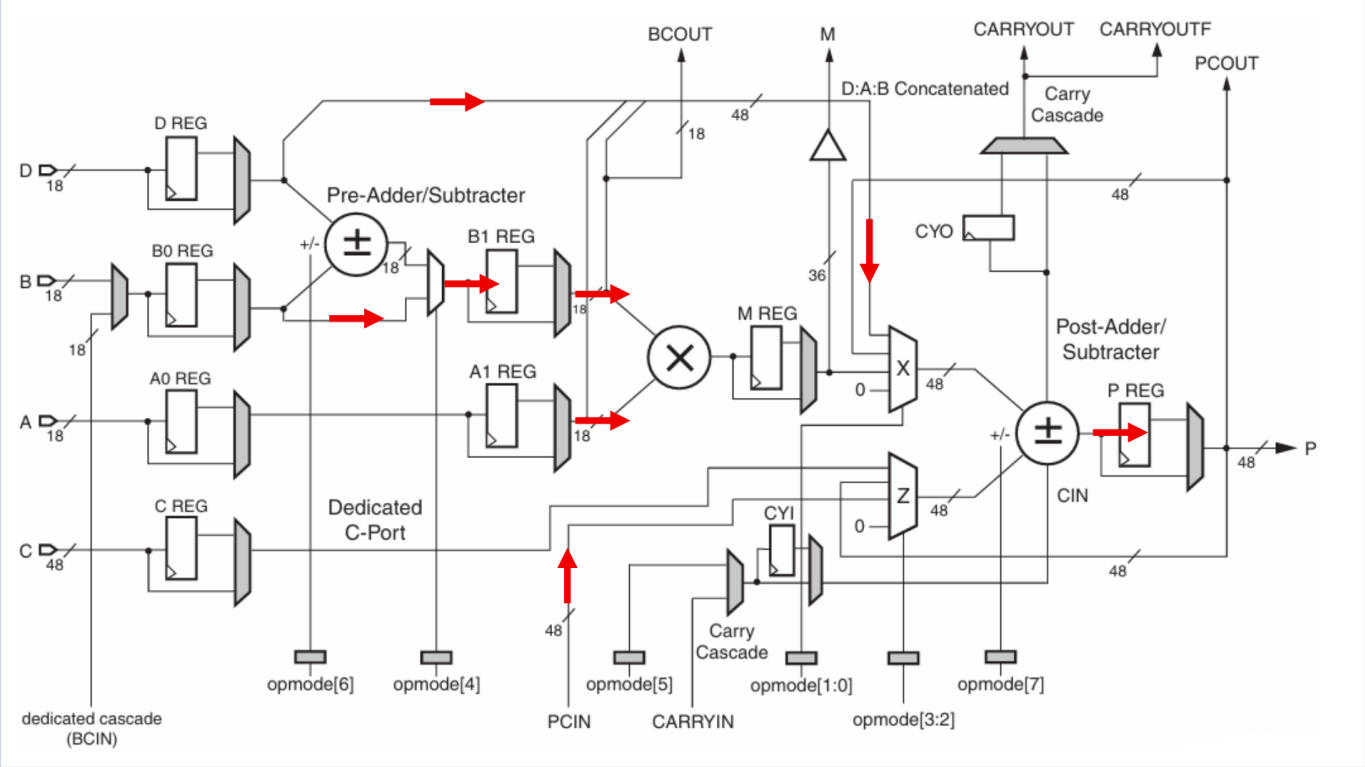


Figure 14. Path 4 data flow.

Testbench code, part 1: setup and reset test (2.1)

```
module pro_1_tb #(
    parameter WIDTH_ABD = 18,
    parameter WIDTH_C = 48,
    parameter WIDTH_OP = 8,
    parameter WIDTH_P = 48,
    parameter WIDTH_M = 36
)();
    reg [WIDTH_ABD-1:0] A,B,D,BCIN;
    reg [WIDTH_C-1:0] C,PCIN;
    reg [WIDTH_OP-1:0] OPMODE;
    reg CARRYIN;
    reg CEA,CEB,CEC,CED,CEOPMODE,CEP,CECARRYIN,CEM,clk,
    RSTA,RSTB,RSTC,RSTD,RSTOPMODE,RSTP,RSTCARRYIN,RSTM;
    wire [WIDTH_ABD-1:0] BCOUT;
    wire [WIDTH_M-1:0] M;
    wire [WIDTH_P-1:0] P,PCOUT;
    wire CARRYOUT,CARRYOUTF;
    pro_1 dut(
        A,B,D,BCIN,
        C,PCIN,
        OPMODE,CARRYIN,
        CEA,CEB,CEC,CED,CEOPMODE,CEP,CECARRYIN,CEM,clk,
        RSTA,RSTB,RSTC,RSTD,RSTOPMODE,RSTP,RSTCARRYIN,RSTM,
        BCOUT,M,P,PCOUT,
        CARRYOUT,CARRYOUTF
    );
    initial begin
        clk = 0 ;
        forever begin
            #1 clk = ~clk;
        end
    end
    initial begin
        //2.1
        RSTA=1;
        RSTB=1;
        RSTC=1;
        RSTD=1;
        RSTOPMODE=1;
        RSTP=1;
        RSTCARRYIN=1;
        RSTM=1;
        repeat(7) begin
            A=$random;
            B=$random;
            D=$random;
            BCIN=$random;
            C=$random;
            PCIN=$random;
            OPMODE=$random;
            CARRYIN=$random;
            CEA=$random;
            CEB=$random;
            CEC=$random;
            CED=$random;
            CEOPMODE=$random;
            CEP=$random;
            CECARRYIN=$random;
            CEM=$random;
            @(negedge clk);
            if (BCOUT != 'b0) begin
                $display(" the  BCOUT expected is %b but the output is %b " , 'b0,BCOUT);
            end
            if (M != 'b0) begin
                $display(" the  M expected is %b but the output is %b " , 'b0,M);
            end
            if (P != 'b0) begin
                $display(" the  P expected is %b but the output is %b " , 'b0,P);
            end
            if (PCOUT != 'b0) begin
                $display(" the  PCOUT expected is %b but the output is %b " , 'b0,PCOUT);
            end
            if (CARRYOUT != 'b0) begin
                $display(" the  CARRYOUT expected is %b but the output is %b " , 'b0,CARRYOUT);
            end
            if (CARRYOUTF != 'b0) begin
                $display(" the  CARRYOUTF expected is %b but the output is %b " , 'b0,CARRYOUTF);
            end
        end
        RSTA=0;
        RSTB=0;
        RSTC=0;
        RSTD=0;
        RSTOPMODE=0;
        RSTP=0;
        RSTCARRYIN=0;
        RSTM=0;
        CEA=1;
        CEB=1;
        CEC=1;
        CED=1;
        CEOPMODE=1;
        CEP=1;
        CECARRYIN=1;
        CEM=1;
        //2.2
```

Figure 15. `pro_1_tb`, *part 1.*

Part 1: setup and reset test

DUT instance and clock generation, then **test 2.1**: every reset is asserted and 7 rounds of random stimulus are applied. All outputs (BCOUT, M, P, PCOUT, CARRYOUT, CARRYOUTF) must stay at zero; any mismatch is printed with `$display`.

Resets are then released and all clock enables set to 1, ready for test 2.2.

Testbench code, part 2: directed tests 2.2 – 2.4

Each test sets OPMODE, repeats 7 times with random BCIN, PCIN and CARRYIN, waits for the pipeline to fill and compares the outputs against expected values.

```
OPMODE=8'b11011101;
        A = 20;
        B = 10;
        C = 350;
        D = 25;
        repeat(7) begin
            BCIN=$random;
            PCIN=$random;
            CARRYIN=$random;
            @(negedge clk);
            @(negedge clk);
            @(negedge clk);
            @(negedge clk);
            if (BCOUT != 'hf) begin
                $display(" the  BCOUT expected is %h but the output is %h " , 'hf,BCOUT);
            end
            if (M != 'h12c) begin
                $display(" the  M expected is %h but the output is %h " , 'h12c,M);
            end
            if (P != 'h32) begin
                $display(" the  P expected is %h but the output is %h " , 'h32,P);
            end
            if (PCOUT != P) begin
                $display(" the  PCOUT & P expected is %h but the PCOUT & P is %h  , %h ",'h32 , PCOUT,P);
            end
            if (CARRYOUT != 'b0) begin
                $display(" the  CARRYOUT expected is %b but the output is %b " , 'b0,CARRYOUT);
            end
            if (CARRYOUTF != CARRYOUT) begin
                $display(" the  CARRYOUTF & CARRYOUT expected is %b but the CARRYOUTF & CARRYOUT is %b  , %b ",'b0 , CARRYOUTF,CARRYOUT);
            end
        end
        //2.3
        OPMODE=8'b00010000;
        repeat(7) begin
            BCIN=$random;
            PCIN=$random;
            CARRYIN=$random;
            @(negedge clk);
            @(negedge clk);
            @(negedge clk);
            if (BCOUT != 'h23) begin
                $display(" the  BCOUT expected is %h but the output is %h " , 'h23,BCOUT);
            end
            if (M != 'h2bc) begin
                $display(" the  M expected is %h but the output is %h " , 'h2bc,M);
            end
            if (P != 'b0) begin
                $display(" the  P expected is %b but the output is %b " , 'b0,P);
            end
            if (PCOUT != P) begin
                $display(" the  PCOUT & P expected is %b but the PCOUT & P is %b  , %b ",'b0, PCOUT,P);
            end
            if (CARRYOUT != 'b0) begin
                $display(" the  CARRYOUT expected is %b but the output is %b " , 'b0,CARRYOUT);
            end
            if (CARRYOUTF != CARRYOUT) begin
                $display(" the  CARRYOUTF & CARRYOUT expected is %b but the CARRYOUTF & CARRYOUT is %b  , %b ",'b0 , CARRYOUTF,CARRYOUT);
            end
        end
        //2.4
        OPMODE=8'b00001010;
        repeat(7) begin
            BCIN=$random;
            PCIN=$random;
            CARRYIN=$random;
            @(negedge clk);
            @(negedge clk);
            @(negedge clk);
            if (BCOUT != 'ha) begin
                $display(" the  BCOUT expected is %h but the output is %h " , 'ha,BCOUT);
            end
            if (M != 'hc8) begin
                $display(" the  M expected is %h but the output is %h " , 'hc8,M);
            end
            if (PCOUT != P) begin
                $display(" the  PCOUT & P expected is %h but the PCOUT & P is %h  , %h ",P, PCOUT,P);
            end
            if (CARRYOUTF != CARRYOUT) begin
                $display(" the  CARRYOUTF & CARRYOUT expected is %h but the CARRYOUTF & CARRYOUT is %h  , %h ",CARRYOUT, CARRYOUTF,CARRYOUT);
            end
        end
```

Figure 16. *Directed tests 2.2 (OPMODE = 8'b11011101), 2.3 (8'b00010000) and 2.4 (8'b00001010).*

Testbench code, part 3: test 2.5 and end of testbench

```
//2.5
        OPMODE=8'b10100111;
        A = 5;
        B = 6;
        C = 350;
        D = 25 ;
        PCIN = 3000;
        repeat(7) begin
            BCIN=$random;
            CARRYIN=$random;
            @(negedge clk);
            @(negedge clk);
            @(negedge clk);
            if (BCOUT != 'h6) begin
                $display(" the  BCOUT expected is %h but the output is %h " , 'h6,BCOUT);
            end
            if (M != 'h1e) begin
                $display(" the  M expected is %h but the output is %h " , 'h1e,M);
            end
            if (P != 'hfe6fffec0bb1) begin
                $display(" the  P expected is %h but the output is %h " , 'hfe6fffec0bb1,P);
            end
            if (PCOUT != P) begin
                $display(" the  PCOUT & P expected is %h but the PCOUT & P is %h  , %h ",'hfe6fffec0bb1, PCOUT,P);
            end
            if (CARRYOUT != 1) begin
                $display(" the  CARRYOUT expected is %b but the output is %b " , 1,CARRYOUT);
            end
            if (CARRYOUTF != CARRYOUT) begin
                $display(" the  CARRYOUTF & CARRYOUT expected is %b but the CARRYOUTF & CARRYOUT is %b  , %b ",1 , CARRYOUTF,CARRYOUT);
            end
        end
        $stop;
    end
endmodule //pro_1_tb
```

Figure 17. *Test 2.5 (OPMODE = 8'b10100111, A = 5, B = 6, C = 350, D = 25, PCIN = 3000), followed by* `$stop`.

06 Elaboration

Vivado RTL analysis

Schematic

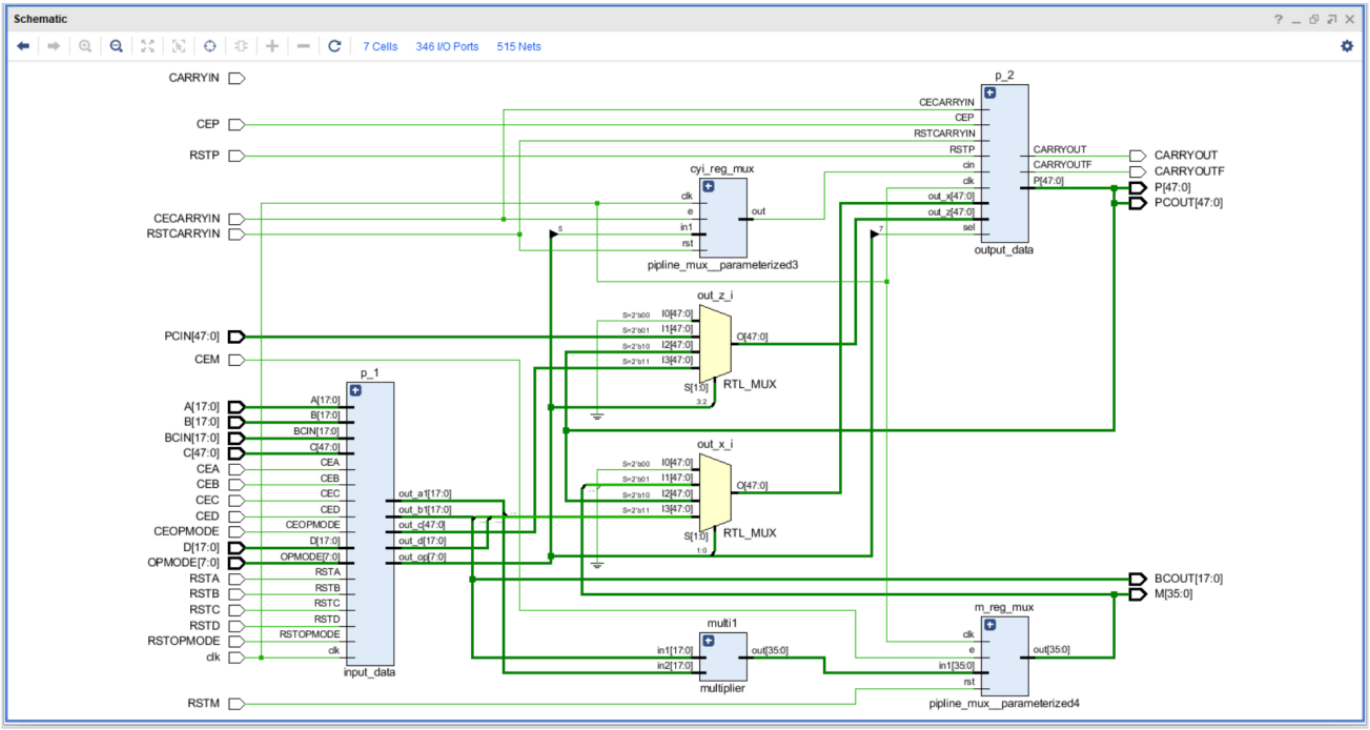


Figure 18. Elaborated design schematic (7 cells, 346 I/O ports, 515 nets).

Messages

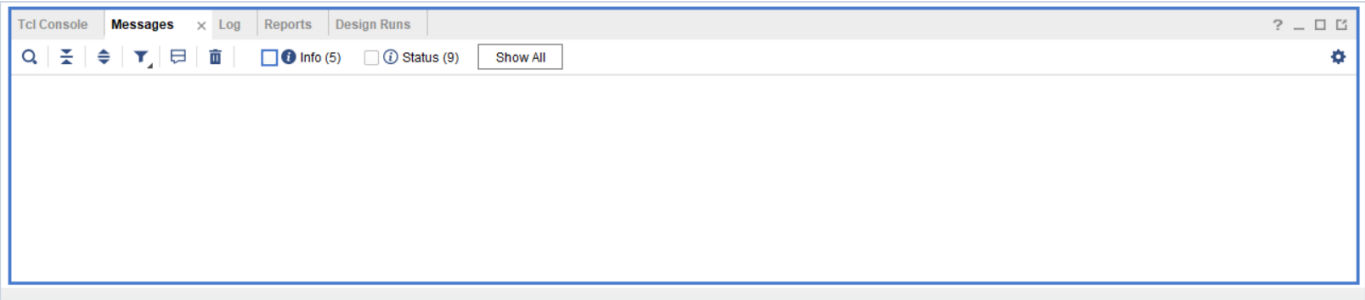


Figure 19. Messages tab after elaboration.

07 Synthesis

Vivado synthesised design

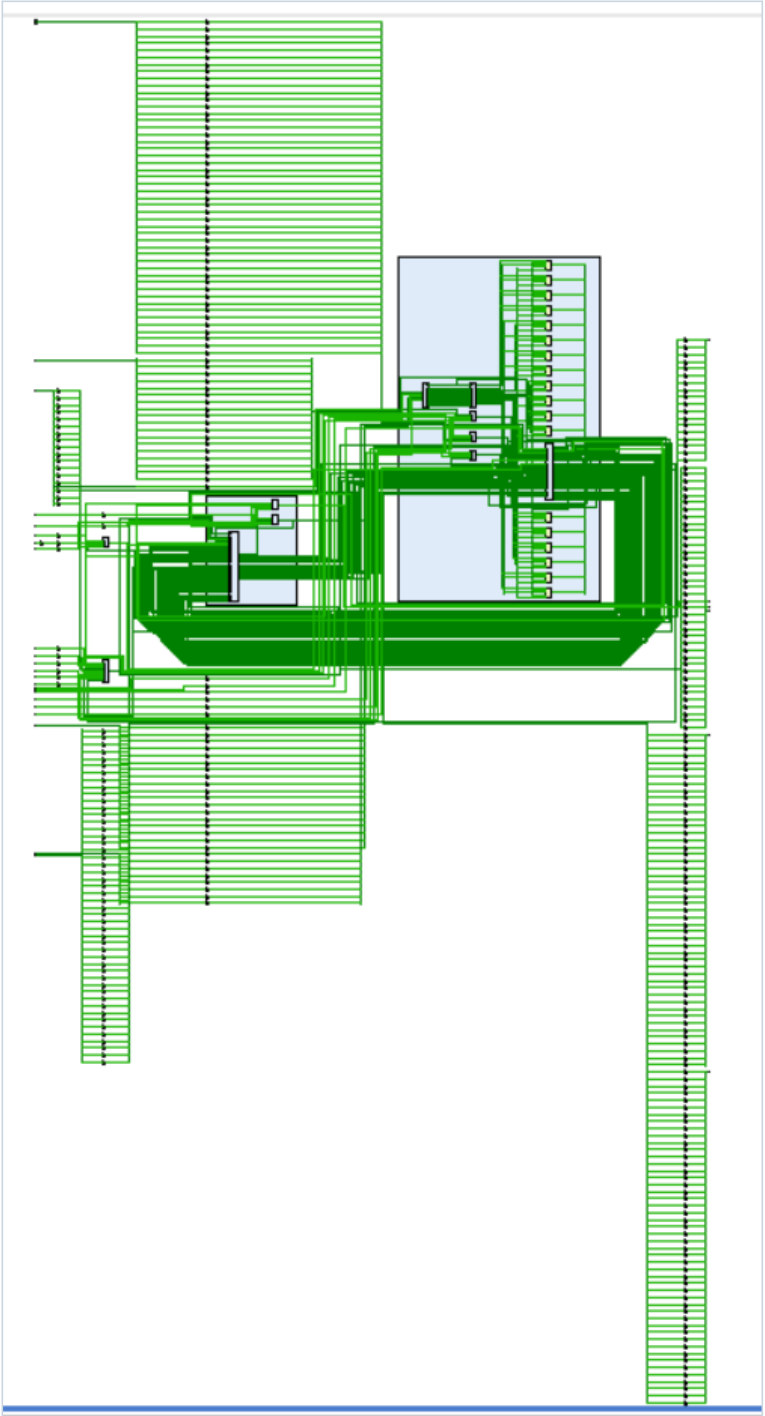


Figure 20. Synthesised schematic.

Post-synthesis summary

Metric	Value
Cells	359
I/O ports	346
Nets	1883
Slice LUTs	273
Slice registers	160
DSP blocks	1
WNS	5.216 ns
WHS	0.182 ns
WPWS	4.500 ns

All user-specified timing constraints are met.

Timing and utilisation

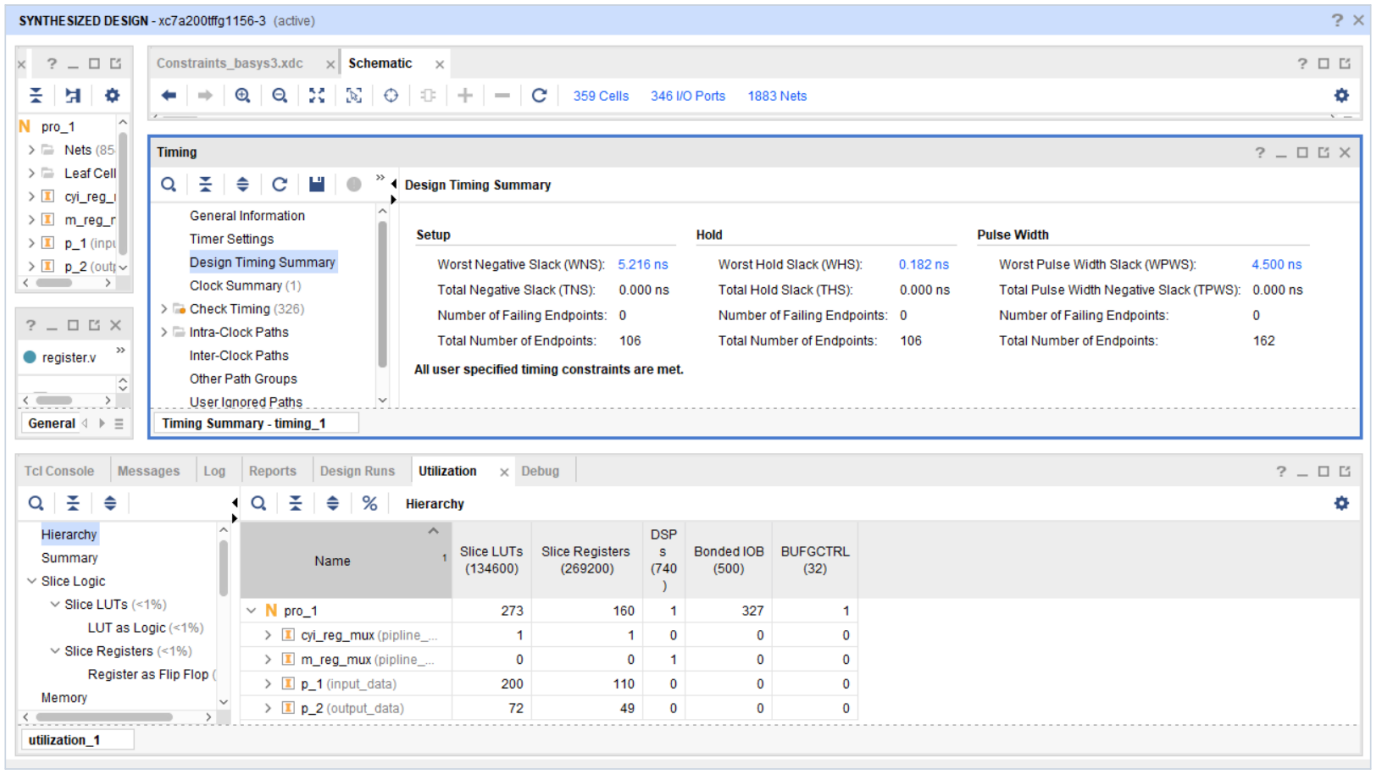


Figure 21. *Post-synthesis design timing summary and hierarchical utilisation.*

08 Implementation

Vivado placed and routed design

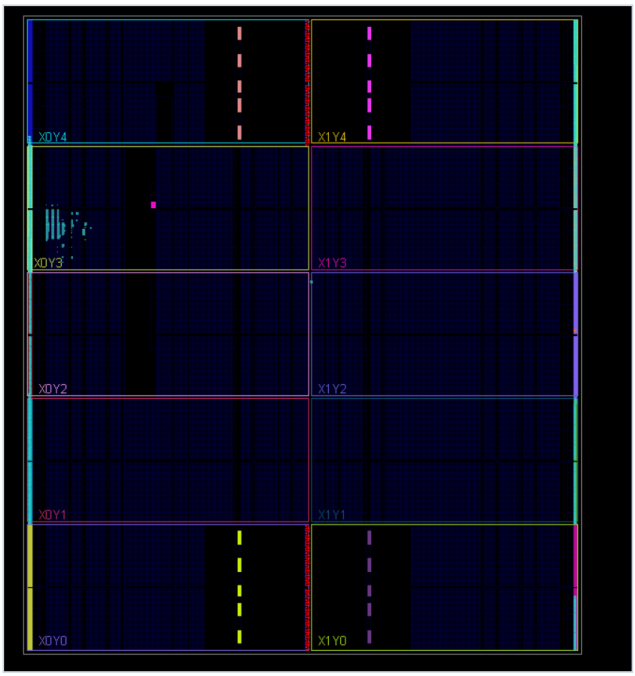


Figure 22. Device view after implementation.

Post-implementation summary

Metric	Value
Slice LUTs	273
Slice registers	179
Slices	114
DSP blocks	1
WNS	3.716 ns
WHS	0.262 ns
WPWS	4.500 ns

All user-specified timing constraints are met.

Timing and utilisation

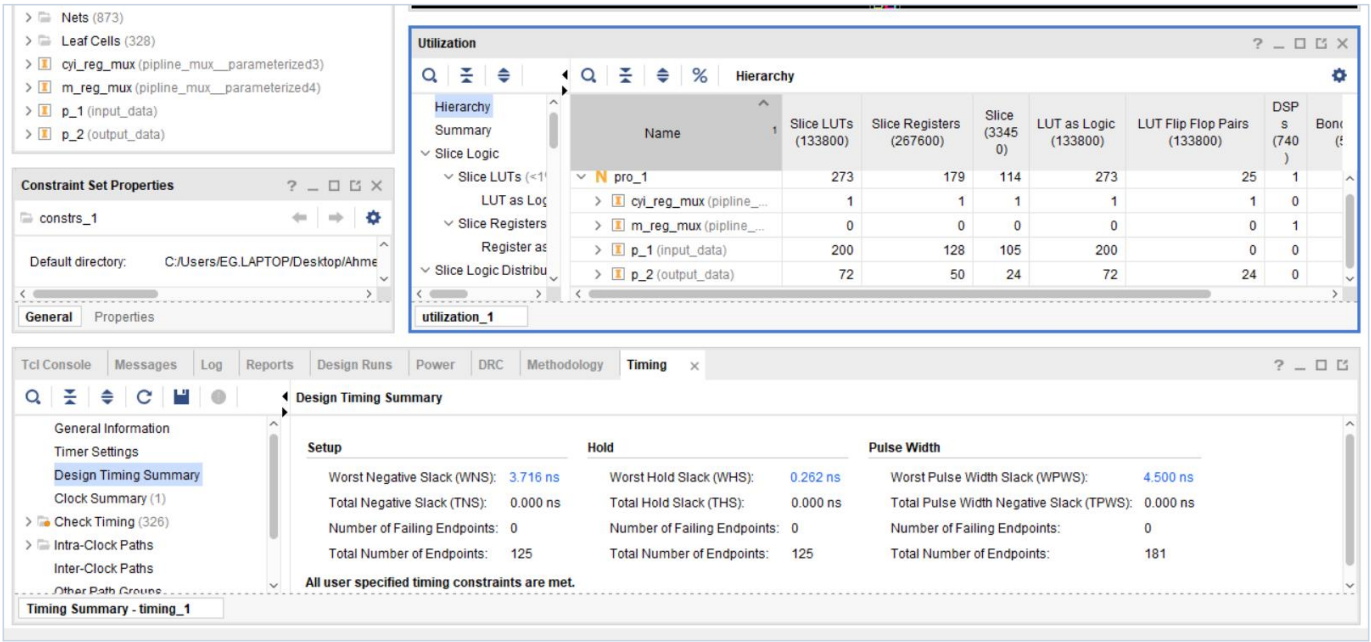


Figure 23. Post-implementation utilisation and design timing summary.

Constraint file

```
## Clock signal
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

Figure 24. Clock constraint (10.00 ns period) in the XDC file.

09 Linting

STARC rule check

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference

Figure 25. Linting report: no violations.